

Berliner Hochschule für Technik

Bachelorarbeit

Konzeption und Implementierung einer interaktiven Visualisierung präemptiver Scheduling-Verfahren für die Lehre

Marten Kirschenhofer
Matrikelnummer: 833825
Studiengang: Medieninformatik

Betreuung: Prof. Dr. Name
Abgabedatum: TT.MM.JJJJ

Vorwort

Diese Bachelorarbeit entstand im Studiengang Medieninformatik an der Berliner Hochschule für Technik. Sie behandelt die Konzeption und Implementierung einer interaktiven Anwendung zur Veranschaulichung präemptiver CPU-Scheduling-Verfahren.

Das Vorwort wird in der Endfassung bei Bedarf um persönliche Danksagungen und organisatorische Hinweise ergänzt.

Contents

Vorwort	ii
List of Figures	v
List of Tables	vi
Abkürzungsverzeichnis	vii
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielsetzung der Arbeit	1
1.3 Forschungsfrage und Vorgehensweise	2
1.4 Aufbau der Arbeit	2
2 Grundlagen	3
2.1 Betriebssysteme und Prozessverwaltung	3
2.2 CPU-Scheduling in Betriebssystemen	3
2.3 Präemptive Scheduling-Verfahren	3
2.4 Überblick über die betrachteten Algorithmen	4
2.5 Kriterien zur Bewertung von Scheduling-Verfahren	4
3 Didaktische Grundlagen interaktiver Visualisierungen	6
3.1 Stand der Forschung zu Scheduling-Visualisierungen	6
3.2 Lernen mit digitalen Medien	7
3.3 Multimediales und visuelles Lernen	7
3.4 Didaktischer Nutzen von Simulationen	7
3.5 Visualisierung komplexer Abläufe im Lernkontext	8
3.6 Anforderungen an die didaktische Aufbereitung von Scheduling-Prozessen	8
4 Grundlagen der Mensch-Computer-Interaktion	9
4.1 Ziele der Mensch-Computer-Interaktion	9
4.2 Usability und Nutzerfreundlichkeit	9
4.3 Interaktionsgestaltung für Lernanwendungen	9
4.4 Rückmeldung, Transparenz und Verständlichkeit	10
4.5 Anforderungen an die Benutzungsoberfläche des Visualizers	10
5 Anforderungen und Konzeption des Scheduling-Visualizers	12
5.1 Zielgruppe und Einsatzkontext	12
5.2 Fachliche Anforderungen	12
5.3 Didaktische Anforderungen	13
5.4 Funktionale Anforderungen	13
5.5 Nicht-funktionale Anforderungen	14
5.6 Konzeption der Simulationslogik	14

5.7	Konzeption der Visualisierung	14
5.8	Konzeption der Benutzungsoberfläche	15
6	Implementierung	17
6.1	Technische Grundlagen und verwendete Technologien	17
6.2	Architektur der Anwendung	17
6.3	Modellierung der Scheduling-Verfahren	18
6.4	Umsetzung der Simulation	18
6.5	Umsetzung der Visualisierung	19
6.6	Umsetzung der Interaktionsmechanismen	19
6.7	Besondere technische Herausforderungen	20
7	Evaluation	21
7.1	Zielsetzung der Evaluation	21
7.2	Evaluationsdesign	21
7.3	Durchführung der Evaluation	22
7.4	Auswertung der Ergebnisse	23
7.5	Diskussion der Ergebnisse	25
8	Fazit und Ausblick	27
8.1	Zusammenfassung der Ergebnisse	27
8.2	Beantwortung der Forschungsfrage	27
8.3	Ausblick auf Weiterentwicklungen	28
9	Anhang	31

List of Figures

5.1	Szenario-Generator zur Definition von Prozessdaten und Simulationsparametern.	15
6.1	Fokusansicht mit Gantt-Zeitachse, Stack-Visualisierung und Kennzahlen. . .	19
7.1	Vergleichsübersicht mit zwei Round-Robin-Läufen und Bewertungsmatrix. .	24
7.2	Bewertungsmatrix zur gewichteten Einordnung mehrerer Simulationsläufe. .	24

List of Tables

7.1	Basisszenarien für die Evaluation	22
7.2	Messwerte der Parametersensitivität	23
7.3	Erste Messwerte der Basisszenarien	25

Abkürzungsverzeichnis

CPU Central Processing Unit

HCI Human-Computer Interaction

MLFQ Multilevel Feedback Queue

RR Round Robin

SRTF Shortest Remaining Time First

1 Einleitung

1.1 Motivation und Problemstellung

CPU-Scheduling ist ein zentrales Konzept moderner Betriebssysteme, da es bestimmt, welcher Prozess zu welchem Zeitpunkt Rechenzeit erhält. Obwohl die theoretischen Regeln einzelner Verfahren in Lehrbüchern klar beschrieben sind, ist das dynamische Verhalten in realistischen Lastsituationen für Studierende häufig schwer nachzuvollziehen [10, 12].

Insbesondere präemptive Verfahren erzeugen viele zeitkritische Zustandswechsel. Dazu gehören Unterbrechungen laufender Prozesse, erneute Einplanung sowie wechselnde Prioritätslagen. Diese Dynamik lässt sich in statischen Tabellen oder rein textlichen Beschreibungen nur eingeschränkt vermitteln. Das führt dazu, dass Lernende zwar Definitionswissen erwerben, aber Schwierigkeiten bei der Interpretation konkreter Ablaufmuster haben.

Vor diesem Hintergrund besteht die Problemstellung dieser Arbeit darin, ein didaktisch geeignetes und technisch belastbares Werkzeug bereitzustellen, das algorithmische Entscheidungen sichtbar macht und Verfahren unter identischen Bedingungen vergleichbar darstellt.

1.2 Zielsetzung der Arbeit

Ziel der Arbeit ist die Konzeption und Implementierung einer interaktiven Webanwendung zur Veranschaulichung präemptiver Scheduling-Verfahren. Die Anwendung soll:

- ausgewählte Verfahren fachlich korrekt simulieren,
- deren zeitlichen Ablauf transparent visualisieren,
- und den systematischen Vergleich zwischen Verfahren anhand einheitlicher Kennzahlen ermöglichen.

Darüber hinaus verfolgt die Arbeit eine didaktische Zielsetzung: Die Anwendung soll den Übergang von theoretischer Regelbeschreibung zu beobachtbarer Verhaltensanalyse erleichtern. Dies wird durch interaktive Steuerung, reproduzierbare Szenarien und konsistente Metrikdarstellung unterstützt [5, 7].

1.3 Forschungsfrage und Vorgehensweise

Die zentrale Forschungsfrage lautet:

Wie kann eine interaktive Visualisierung präemptiver Scheduling-Verfahren so konzipiert und implementiert werden, dass sie fachliche Korrektheit, didaktische Nachvollziehbarkeit und vergleichende Analyse vereint?

Zur Beantwortung dieser Frage wird ein mehrstufiges Vorgehen gewählt:

1. Aufarbeitung theoretischer Grundlagen zu Betriebssystemen, Scheduling und Bewertungsmetriken.
2. Ableitung didaktischer und HCI-bezogener Anforderungen an die Anwendung.
3. Konzeption der Simulations- und Visualisierungsarchitektur.
4. Implementierung als Webanwendung mit typischerem Simulationskern.
5. Quantitative Evaluation über reproduzierbare Szenarien und Metrikvergleiche.

Das methodische Grundprinzip besteht darin, alle Auswertungen auf einem deterministischen Simulationsmodell aufzubauen, um Reproduzierbarkeit und wissenschaftliche Nachvollziehbarkeit sicherzustellen [2].

1.4 Aufbau der Arbeit

Die Arbeit ist in acht Kapitel gegliedert. Kapitel 2 stellt die theoretischen Grundlagen zu Scheduling und Bewertungsmetriken dar. Kapitel 3 und Kapitel 4 behandeln didaktische Grundlagen sowie Prinzipien der Mensch-Computer-Interaktion als Anforderungsbasis.

Kapitel 5 konkretisiert darauf aufbauend die fachlichen, didaktischen, funktionalen und nicht-funktionalen Anforderungen und beschreibt die Konzeption der Anwendung. Kapitel 6 dokumentiert die Implementierung mit Fokus auf Architektur, Simulationslogik, Visualisierung und Interaktionsmechanismen.

Kapitel 7 präsentiert das quantitative Evaluationsdesign und diskutiert die Ergebnisse. Kapitel 8 fasst die wesentlichen Erkenntnisse zusammen, beantwortet die Forschungsfrage und skizziert Weiterentwicklungsmöglichkeiten.

2 Grundlagen

Dieses Kapitel beschreibt die theoretischen Grundlagen, auf denen Konzeption, Implementierung und Evaluation der Anwendung aufbauen.

2.1 Betriebssysteme und Prozessverwaltung

Betriebssysteme koordinieren den Zugriff auf Hardware-Ressourcen und stellen Ausführungsumgebungen für Programme bereit. Eine Kernaufgabe ist dabei die Verwaltung konkurrierender Prozesse, die um CPU-Zeit, Speicher und weitere Ressourcen konkurrieren [10, 12].

Prozesse durchlaufen typischerweise mehrere Zustände, etwa bereit, laufend und blockiert. Das Betriebssystem steuert Übergänge zwischen diesen Zuständen und bestimmt durch Scheduling-Entscheidungen, welcher bereite Prozess als nächstes ausgeführt wird. Diese Entscheidungen wirken sich direkt auf wahrgenommene Systemreaktion und Gesamtdurchlauf aus.

2.2 CPU-Scheduling in Betriebssystemen

CPU-Scheduling beschreibt die Auswahlstrategie für die Vergabe von Prozessorzeit an bereite Prozesse. Ziel ist ein sinnvoller Ausgleich zwischen konkurrierenden Kriterien wie kurzer Reaktionszeit, niedriger Wartezeit, hoher Auslastung und fairer Behandlung [8, 10].

In Mehrprogrammsystemen ist Scheduling eine kontinuierliche Entscheidung unter Unsicherheit, da Ankunftszeitpunkte und Prozessverhalten variieren. Deshalb sind Scheduling-Verfahren typischerweise als Heuristiken formuliert, deren Eignung vom Lastprofil und vom Systemziel abhängt.

2.3 Präemptive Scheduling-Verfahren

Präemptive Verfahren erlauben es dem Betriebssystem, einen laufenden Prozess aktiv zu unterbrechen und einen anderen Prozess einzuplanen. Dies verbessert in vielen Kontexten die Reaktionsfähigkeit, verursacht jedoch zusätzlichen Verwaltungsaufwand durch Kontextwechsel [12, 10].

Der zentrale Unterschied zu nicht-präemptiven Strategien besteht darin, dass Entscheidungen nicht nur bei Abschluss oder Blockierung eines Prozesses getroffen werden, sondern auch während seiner Ausführung. Dadurch steigt die Steuerbarkeit, gleichzeitig aber auch die Komplexität von Analyse und Visualisierung.

2.4 Überblick über die betrachteten Algorithmen

2.4.1 Last Come First Served

Last Come First Served (LCFS) priorisiert neu eingetroffene Prozesse. In der präemptiven Variante kann ein neu ankommender Prozess den aktuell laufenden Prozess sofort verdrängen. Das Verfahren ist anschaulich, kann aber unter ungünstigen Lastprofilen zu langen Wartezeiten früher eingetroffener Prozesse führen.

2.4.2 Round Robin

Round Robin (RR) weist jedem Prozess ein Zeitquantum zu. Nach Ablauf des Quantums wird der Prozess, falls noch nicht abgeschlossen, wieder in die Bereitwarteschlange eingeordnet. RR gilt als faires Grundverfahren für zeitscheibenbasierte Systeme, da alle Prozesse regelmässig CPU-Zeit erhalten [10].

2.4.3 Strict Priority

Strict Priority wählt stets den Prozess mit der höchsten Priorität aus. Trifft ein noch höher priorisierter Prozess ein, kann eine sofortige Präemption stattfinden. Vorteilhaft ist die gezielte Bevorzugung wichtiger Aufgaben; problematisch ist jedoch das Risiko von Starvation für niedrig priorisierte Prozesse [12].

2.4.4 Multilevel Feedback Queue

Multilevel Feedback Queue (MLFQ) kombiniert mehrere Warteschlangenebenen mit unterschiedlichen Prioritäten und Zeitquanten. Prozesse können in niedrigere Ebenen verschoben werden, wenn sie CPU-intensiv sind, während kurz laufende oder interaktive Prozesse in höheren Ebenen profitieren. Dadurch entsteht ein adaptives Verhalten, das auf heterogene Lastprofile reagieren kann [10, 6].

2.5 Kriterien zur Bewertung von Scheduling-Verfahren

Die Bewertung von Scheduling-Verfahren erfolgt über klar definierte Kennzahlen. Diese sind entscheidend, um Algorithmen nicht nur qualitativ, sondern auch quantitativ zu

vergleichen [8].

2.5.1 Wartezeit

Die Wartezeit beschreibt die Zeitspanne, die ein Prozess im Zustand „bereit“ verbringt, ohne ausgeführt zu werden. Niedrige durchschnittliche Wartezeiten sind insbesondere in Systemen mit vielen kurzen Aufgaben wünschenswert.

2.5.2 Durchlaufzeit

Die Durchlaufzeit (Turnaround Time) ist die Zeit vom Eintreffen eines Prozesses bis zu dessen Abschluss. Sie bildet den Gesamtablauf aus Sicht eines Auftrags ab und ist eine zentrale Effizienzmetrik für batch-orientierte Lastprofile.

2.5.3 Reaktionszeit

Die Reaktionszeit beschreibt die Zeit vom Eintreffen eines Prozesses bis zu seiner ersten CPU-Zuteilung. Diese Kennzahl ist besonders relevant für interaktive Systeme, in denen frühes Feedback wichtiger sein kann als minimale Gesamtdurchlaufzeit [10, 12].

Ergänzend können Kontextwechselhäufigkeit, Präemptionsanzahl, Fairnessmasse und CPU-Leerlaufanteile betrachtet werden, um Zielkonflikte zwischen Responsivität, Stabilität und Ressourceneffizienz sichtbar zu machen.

3 Didaktische Grundlagen interaktiver Visualisierungen

3.1 Stand der Forschung zu Scheduling-Visualisierungen

Algorithmusvisualisierungen werden seit mehreren Jahrzehnten eingesetzt, um abstrakte Abläufe durch sichtbare Zustandsänderungen und zeitliche Beziehungen verständlicher zu machen. Das AlgoViz-Portal dokumentiert eine Vielzahl unterschiedlicher Visualisierungsansätze und verdeutlicht damit die Breite des Forschungs- und Anwendungsfeldes [1]. Für die vorliegende Arbeit sind insbesondere Werkzeuge relevant, die Scheduling-Entscheidungen in Form von Zeitachsen, Prozesszuständen oder Gantt-Diagrammen darstellen.

Neben allgemeinen Portalen existieren einzelne offene Implementierungen von Scheduling-Simulatoren. Beispiele für MLFQ-orientierte Ansätze zeigen, dass Prozesswarteschlangen, Zeitscheiben und Ausführungsabschnitte grundsätzlich interaktiv oder zumindest programmatisch visualisiert werden können [9, 13]. Diese Ansätze sind jedoch häufig auf ein einzelnes Verfahren, eine einfache Demonstration oder eine konkrete Implementierung beschränkt. Eine einheitliche Gegenüberstellung mehrerer Verfahren unter identischen Eingaben und mit gemeinsam berechneten Kennzahlen steht dabei nicht im Mittelpunkt.

Die Forschung zur Algorithmusvisualisierung liefert ergänzend Hinweise zur Gestaltung solcher Lernwerkzeuge. Meta-Studien zeigen, dass Visualisierungen allein nicht automatisch zu besseren Lernergebnissen führen. Entscheidend ist, wie aktiv Lernende einbezogen werden und ob sie Abläufe untersuchen, verändern und mit ihren eigenen Erwartungen vergleichen können [5, 7]. Gestaltungsprinzipien der Softwarevisualisierung betonen deshalb die Verbindung von sichtbarem Ablauf, Interaktion und verständlicher Rückmeldung [11].

Aus diesem Forschungsstand ergibt sich die Positionierung der vorliegenden Arbeit: Der Scheduling-Visualizer verbindet mehrere präemptive Verfahren mit einer gemeinsamen, deterministischen Simulationsgrundlage, einer schrittweise steuerbaren Zeitachsenvisualisierung und vergleichbaren Metriken. Der Schwerpunkt liegt damit nicht auf der Entwicklung eines weiteren isolierten Simulators, sondern auf der didaktisch begründeten Verbindung von fachlicher Simulation, interaktiver Exploration und vergleichender Analyse.

3.2 Lernen mit digitalen Medien

Digitale Lernmedien ermöglichen es, fachliche Inhalte nicht nur rezeptiv aufzunehmen, sondern aktiv zu erkunden. Gerade in der Informatiklehre ist dieser Unterschied relevant, weil viele Konzepte erst durch das Zusammenspiel mehrerer Faktoren verständlich werden. Dazu gehören im Kontext von Scheduling insbesondere Zeitverlauf, Zustandswechsel und konkurrierende Prozesse. Die Wirksamkeit interaktiver Visualisierungsansätze wurde in der Forschung wiederholt untersucht und als lernförderlich eingeordnet [5, 7, 11].

Interaktive Anwendungen unterstützen diesen Lernprozess, indem sie unmittelbare Rückmeldungen liefern. Lernende können Parameter variieren, Effekte beobachten und ihre Annahmen direkt überprüfen. Damit verschiebt sich die Lernaktivität von einer reinen Wissensaufnahme hin zu einem iterativen Verständnisaufbau.

3.3 Multimediales und visuelles Lernen

Scheduling-Verfahren sind abstrakt, weil ihre Logik nicht unmittelbar sichtbar ist. Ob ein Verfahren als fair, effizient oder reaktionsschnell wahrgenommen wird, zeigt sich erst im Verlauf vieler Zeitschritte. Eine visuelle Darstellung auf einer Zeitachse reduziert diese Abstraktion, indem sie zeitliche Relationen explizit macht.

Insbesondere Gantt-Diagramme sind geeignet, da sie Prozessausführung, Wartephasen und Unterbrechungen in einem gemeinsamen Koordinatensystem zeigen. Das erleichtert die mentale Verknüpfung zwischen formaler Algorithmusbeschreibung und beobachtbarem Laufzeitverhalten.

3.4 Didaktischer Nutzen von Simulationen

Simulationen haben im Lernkontext den Vorteil, dass sie komplexe Systemdynamiken kontrollierbar machen. Lernende müssen kein reales Betriebssystem modifizieren, um Scheduling-Effekte zu verstehen. Stattdessen können sie reproduzierbare Szenarien durchspielen und unterschiedliche Algorithmen unter identischen Voraussetzungen vergleichen. Diese Vorgehensweise entspricht dem methodischen Nutzen diskreter Ereignissimulation für Analyse- und Lernzwecke [2].

Für die vorliegende Arbeit ist besonders relevant, dass die Simulation deterministisch arbeitet. Dadurch bleiben Ergebnisse bei gleichen Eingaben stabil. Diese Reproduzierbarkeit ist didaktisch hilfreich, weil Diskussionen in Übungen auf dieselben Beobachtungen Bezug nehmen können.

3.5 Visualisierung komplexer Abläufe im Lernkontext

Präemptive Verfahren erzeugen häufige Kontextwechsel und damit diskontinuierliche Ablaufmuster. In statischen Darstellungen gehen diese Übergänge oft verloren oder sind nur schwer nachzuvollziehen. Eine interaktive Visualisierung kann hingegen Schritt für Schritt zeigen,

- wann ein Prozess startet,
- wann und warum er unterbrochen wird,
- wann er erneut eingeplant wird,
- und wie sich diese Entscheidungen auf Kennzahlen auswirken.

Damit wird aus einer isolierten Theorieerklärung ein nachvollziehbarer Entscheidungsprozess. Genau dieser Aspekt ist für die Lehre zentral, weil Lernende nicht nur Ergebnisse sehen, sondern den Weg dorthin verstehen sollen.

3.6 Anforderungen an die didaktische Aufbereitung von Scheduling-Prozessen

Aus den genannten Grundlagen lassen sich für den Scheduling-Visualizer konkrete didaktische Anforderungen ableiten:

- **Transparenz:** Alle relevanten Zustandsänderungen müssen sichtbar und erklärbar sein.
- **Vergleichbarkeit:** Verschiedene Algorithmen sollen auf identischen Szenarien gegenübergestellt werden.
- **Kontrollierbarkeit:** Nutzende brauchen eine Steuerung für Start, Pause, Schrittbetrieb und Zeitsprünge.
- **Reduktion kognitiver Last:** Die Darstellung muss auf wesentliche Signale fokussieren und visuelle Überfrachtung vermeiden.
- **Anschlussfähigkeit:** Fachbegriffe und Messgrößen müssen konsistent mit den theoretischen Grundlagen verwendet werden.

Diese Anforderungen bilden die Brücke zwischen didaktischer Zielsetzung und technischer Umsetzung. Sie werden in den folgenden Kapiteln bei Konzeption, Implementierung und Evaluation wieder aufgegriffen. Zugleich verknüpfen sie fachliche Kriterien aus der Scheduling-Literatur mit den Anforderungen didaktischer Aufbereitung [12, 10].

4 Grundlagen der Mensch-Computer-Interaktion

4.1 Ziele der Mensch-Computer-Interaktion

Mensch-Computer-Interaktion beschreibt die Gestaltung digitaler Systeme so, dass sie für Menschen verständlich, effizient und verlässlich nutzbar sind. Bei einer Lernanwendung wie dem Scheduling-Visualizer steht nicht allein die funktionale Korrektheit im Vordergrund, sondern auch die Qualität der Interaktion.

Ziel ist eine Oberfläche, die Nutzerinnen und Nutzer sicher durch den Lernprozess führt: Eingaben sollen nachvollziehbar sein, Zustandsänderungen klar rückgemeldet werden, und Bedienhandlungen müssen ohne unnötige Hürden ausführbar bleiben.

4.2 Usability und Nutzerfreundlichkeit

Usability umfasst die Frage, wie wirksam, effizient und zufriedenstellend ein System in einem konkreten Nutzungskontext bedient werden kann. Für den Scheduling-Visualizer bedeutet das unter anderem. Für die spätere Bewertung kann dabei unter anderem auf etablierte Verfahren wie die System Usability Scale zurückgegriffen werden [3]:

- Prozesse und Parameter sollen mit möglichst wenigen Schritten erfasst werden können,
- Simulationszustände müssen eindeutig erkennbar sein,
- und Fehlbedienungen sollen durch klares Feedback schnell korrigierbar bleiben.

Eine hohe Nutzerfreundlichkeit wirkt hier direkt auf den Lernerfolg, weil kognitive Ressourcen nicht in Bedienproblemen gebunden werden, sondern für das fachliche Verständnis verfügbar bleiben.

4.3 Interaktionsgestaltung für Lernanwendungen

Interaktionsgestaltung im Lernkontext folgt dem Prinzip der didaktischen Unterstützung: Das Interface soll Lernziele erleichtern, nicht verdecken. Im vorliegenden Fall betrifft das

insbesondere die Steuerung des Zeitverlaufs und den Vergleich verschiedener Scheduling-Strategien.

Sinnvolle Interaktion entsteht dabei durch eine klare Aufgabenteilung zwischen Eingabe, Steuerung und Ergebnisdarstellung. Wenn diese Bereiche konsistent strukturiert sind, können Nutzende schneller zwischen Experimentieren und Interpretieren wechseln.

4.4 Rückmeldung, Transparenz und Verständlichkeit

Ein zentrales HCI-Prinzip ist die Sichtbarkeit des Systemzustands. Im Scheduling-Visualizer müssen daher wesentliche Informationen permanent oder mit minimalem Aufwand abrufbar sein. Gleichzeitig sind Anforderungen an Barrierefreiheit früh in die Interaktionsgestaltung zu integrieren, damit zentrale Funktionen für möglichst viele Nutzende zugänglich bleiben [17]:

- aktueller Simulationszeitpunkt,
- gerade laufender Prozess,
- Warteschlangen- und Prioritätszustände,
- wichtige Ereignisse wie Präemptionen und Kontextwechsel.

Rückmeldungen sollen nicht nur korrekt, sondern auch semantisch klar formuliert sein. Das reduziert Interpretationsfehler und erhöht die Nachvollziehbarkeit von Algorithmusentscheidungen.

4.5 Anforderungen an die Benutzungsoberfläche des Visualizers

Auf Basis der HCI-Analyse ergeben sich konkrete Anforderungen an die Benutzungsoberfläche. Die folgenden Punkte konkretisieren diese Anforderungen:

- **Konsistente Navigations- und Fokusführung:** Interaktive Elemente müssen in logischer Reihenfolge erreichbar sein.
- **Barrierearme Interaktion:** Tastaturbedienung, semantische Beschriftungen und ausreichende Kontraste sind notwendig.
- **Robuste Dialoginteraktion:** Modale Dialoge brauchen klares Fokusmanagement sowie nachvollziehbares Schliessen.
- **Eindeutige Statuskommunikation:** Der Unterschied zwischen laufender, pausierter und abgeschlossener Simulation muss sofort erkennbar sein.
- **Fehlertoleranz:** Nutzende sollen Eingaben korrigieren können, ohne den gesamten Arbeitskontext zu verlieren.

Diese Anforderungen greifen die vorhandenen Erkenntnisse zur Interaktionsqualität auf

und bilden die Grundlage für die anschliessende Konzeption und Implementierung der Anwendung. Damit werden didaktische Zielsetzung und HCI-Qualitätskriterien systematisch zusammengeführt [5, 17].

5 Anforderungen und Konzeption des Scheduling-Visualizers

Dieses Kapitel definiert die fachlichen, didaktischen und technischen Anforderungen der Anwendung und leitet daraus das konzeptionelle Lösungsdesign ab. Im Mittelpunkt steht die Frage, wie präemptive Scheduling-Verfahren fachlich korrekt, didaktisch nachvollziehbar und mit hoher Interaktionsqualität visualisiert werden können. Die konzeptionellen Entscheidungen orientieren sich dabei an etablierten Scheduling-Grundlagen [10, 12, 8], an Erkenntnissen aus der Algorithmusvisualisierung [5, 7] sowie an Anforderungen der Usability und Accessibility [3, 17].

5.1 Zielgruppe und Einsatzkontext

Primäre Zielgruppe sind Studierende in einführenden und vertiefenden Lehrveranstaltungen zu Betriebssystemen. Die Anwendung soll insbesondere in Situationen unterstützen, in denen formale Algorithmusbeschreibungen allein nicht ausreichen, um den zeitlichen Ablauf präemptiver Entscheidungen zu verstehen.

Der Einsatzkontext umfasst sowohl Lehrveranstaltungen mit geführter Demonstration als auch das selbstständige Lernen. Daraus folgen zwei zentrale Rahmenbedingungen: Erstens muss die Anwendung reproduzierbare Ergebnisse liefern, damit Beobachtungen zwischen verschiedenen Nutzenden vergleichbar bleiben. Zweitens braucht sie eine flexible Interaktion, damit sowohl exploratives Ausprobieren als auch gezieltes Nachstellen vorgegebener Szenarien möglich ist.

5.2 Fachliche Anforderungen

Aus fachlicher Sicht muss der Visualizer die betrachteten Verfahren konsistent und korrekt modellieren. Dazu gehören insbesondere LCFS, Round Robin, Strict Priority und MLFQ. Relevante Kernaspekte sind Ankunftszeit, Burst-Zeit, Priorität, Zeitquantum, Präemption und Kontextwechsel [10, 12].

Zusätzlich müssen die wichtigsten Vergleichsmetriken nachvollziehbar berechnet werden. Dazu zählen Wartezeit, Durchlaufzeit, Reaktionszeit, Kontextwechselhäufigkeit und weitere Kenngrößen zur Bewertung von Fairness und Ressourcennutzung [8]. Die konzeptionellen

tionelle Anforderung lautet daher, dass Simulationslogik und Metrikberechnung auf derselben Zustandsbasis arbeiten und keine voneinander getrennten Berechnungspfade verwenden.

5.3 Didaktische Anforderungen

Didaktisch soll die Anwendung nicht nur Ergebnisse anzeigen, sondern den Entscheidungsprozess der Algorithmen sichtbar machen. Lernende sollen erkennen können, warum ein Prozess zu einem bestimmten Zeitpunkt ausgewählt, unterbrochen oder erneut eingeplant wird. Dieser Anspruch folgt direkt aus der Forschung zu lernwirksamer Algorithmusvisualisierung, die insbesondere aktive Interaktion und vergleichende Betrachtung als wirksame Lernhebel beschreibt [5, 7, 11].

Die Konzeption priorisiert deshalb drei didaktische Leitprinzipien:

- **Transparenz der Zustandsübergänge:** Ereignisse wie Start, Präemption und Abschluss müssen nachvollziehbar dargestellt werden.
- **Vergleichbarkeit:** Mehrere Algorithmen sollen unter identischen Eingabedaten gegenübergestellt werden können.
- **Kontrollierbares Lerntempo:** Schrittweises und automatisches Abspielen sollen gleichwertig unterstützt werden.

5.4 Funktionale Anforderungen

Funktional muss die Anwendung den gesamten Ablauf von der Szenariodefinition bis zur Ergebnisinterpretation abdecken. Daraus ergeben sich folgende Kernfunktionen:

- Erfassung und Bearbeitung von Prozessmengen mit relevanten Attributen.
- Auswahl eines Scheduling-Verfahrens und seiner Parameter.
- Start, Pause, Schrittmodus und Zeitsprung innerhalb der Simulation.
- Visualisierung der Ausführungssegmente auf einer Zeitachse.
- Ausgabe vergleichbarer Metriken pro Lauf.
- Gegenüberstellung mehrerer Läufe auf identischem Szenario.

Konzeptionell wird damit ein durchgängiger Workflow vom Eingabemodell zur Auswertung abgebildet. Jede Funktion muss auf denselben Simulationsdaten aufbauen, um Inkonsistenzen zwischen Anzeige und Metrikberechnung zu vermeiden.

5.5 Nicht-funktionale Anforderungen

Nicht-funktionale Anforderungen betreffen vor allem Nachvollziehbarkeit, Robustheit und Nutzbarkeit. Die Anwendung soll deterministisch arbeiten, damit identische Eingaben zu identischen Ergebnissen führen. Dies ist für didaktische Vergleichbarkeit und wissenschaftliche Reproduzierbarkeit zentral [2].

Darüber hinaus gelten Anforderungen an Performance und Bedienqualität: Interaktionen müssen unmittelbar rückmelden, die Darstellung muss auch bei grösseren Szenarien stabil bleiben, und zentrale Funktionen müssen ohne Maus nutzbar sein. Anforderungen an semantische Benennung, Fokusführung und Kontrast orientieren sich an etablierten Accessibility-Kriterien [17]. Für die spätere Nutzerbewertung wird ein standardisiertes Usability-Instrument wie SUS als geeignet betrachtet [3].

5.6 Konzeption der Simulationslogik

Die Simulationslogik wird als deterministischer Kern konzipiert, der diskrete Zeitschritte verarbeitet und Ereignisse in wohldefinierter Reihenfolge erzeugt. Die zentrale Entscheidung besteht darin, Simulationszustand, Ereignisprotokoll, Zeitachsen-Segmente und Metriken gemeinsam aus einem einheitlichen Zustandsmodell abzuleiten. Dadurch bleiben alle Ausgaben konsistent.

Konzeptionell werden dabei zwei Anforderungen verbunden: erstens fachliche Korrektheit der Scheduling-Regeln, zweitens auswertbare Verlaufsdaten für Visualisierung und Evaluation. Das Ereignismodell muss daher nicht nur Endergebnisse liefern, sondern auch Zwischenschritte explizit protokollieren, damit Präemptionen und Kontextwechsel analytisch nachvollzogen werden können [10, 8].

5.7 Konzeption der Visualisierung

Als primäres Visualisierungsmedium wird ein Gantt-artiger Zeitverlauf eingesetzt. Diese Form ist geeignet, da sie parallel Informationen über Prozessdauer, Unterbrechungen und Wartephasen abbildet und damit den Vergleich unterschiedlicher Strategien erleichtert.

Die Visualisierung wird konzeptionell als Ableitung des Simulationskerns verstanden, nicht als eigene Logikinstanz. Das bedeutet: Die Darstellung interpretiert vorhandene Zustands- und Segmentdaten, trifft aber keine eigenständigen Scheduling-Entscheidungen. Diese Trennung reduziert Fehlerquellen und erhöht die Validierbarkeit. Für die Interaktion werden ergänzend Tooltips, Hervorhebungen und Zeitsteuerung vorgesehen, um sowohl globale als auch detailorientierte Analyse zu ermöglichen [5, 11].

5.8 Konzeption der Benutzungsoberfläche

Die Benutzungsoberfläche wird in drei logisch getrennte Bereiche gegliedert: Eingabe und Konfiguration, Simulationssteuerung sowie Ergebnisdarstellung. Diese Struktur soll die mentale Last reduzieren und den Wechsel zwischen Experimentieren und Interpretieren erleichtern.

Die Umsetzung dieser Aufteilung ist in Abbildung 5.1 exemplarisch zu sehen. Der Generator verbindet die Eingabe der Prozessdaten mit einer Vorschau und vordefinierten Szenarien.

Szenario-Generator Schliessen

Titel
Versetzt

Beschreibung
Prozesse treffen nacheinander ein. Unter einem passenden präemptiv

Seed
33

Tick-Grösse
1

Klassisch Versetzt Lang Viele Chaos

SRTF RR-Quantum Strict-Priority-Starvation

MLFQ-Treppe Zufall

Prozesse + Prozess

ID	NAME	ANKUNFT	RECHENZEIT	PRIO	FARBE
A	A	0	6	2	—
B	B	2	4	1	—
C	C	4	3	4	—
D	D	5	2	2	—

Versetzt VORSCHAU
Miniatur aus den aktuellen Prozessdaten

0 2 4 6 8

A 0 → 6

B 2 → 6

C 4 → 7

D 5 → 7

Szenario anwenden

Figure 5.1: Szenario-Generator zur Definition von Prozessdaten und Simulationsparametern.

Zentrale Interaktionsprinzipien sind Konsistenz, Sichtbarkeit des Systemzustands und

fehlertolerante Bedienung. Aktionen müssen eindeutig rückgemeldet werden, insbesondere beim Wechsel zwischen pausiertem und laufendem Zustand. Modale Dialoge und Menüs sind mit klarem Fokusverhalten und semantischer Beschriftung auszustatten, damit die Anwendung auch mit assistiven Technologien nutzbar bleibt [17]. Insgesamt bildet die Oberflächenkonzeption damit die operative Umsetzung der in Kapitel 3 und Kapitel 4 abgeleiteten didaktischen und HCI-Anforderungen.

6 Implementierung

Dieses Kapitel beschreibt die konkrete technische Umsetzung auf Basis der zuvor definierten Anforderungen und der konzeptionellen Architektur. Im Fokus stehen die Trennung zwischen Simulationskern und Darstellung, die typsichere Modellierung der Scheduling-Domäne sowie die Umsetzung interaktiver Analysefunktionen.

6.1 Technische Grundlagen und verwendete Technologien

Die Anwendung wurde als rein clientseitige Webanwendung umgesetzt. Der gewählte Technologie-Stack besteht aus Vue 3, TypeScript und Vite [16, 14]. Diese Kombination unterstützt eine schnelle Entwicklungsiteration und bietet zugleich eine klare Struktur für komponentenbasierte Oberflächen und typsichere Fachlogik.

Vue 3 wird für die komponentenorientierte UI-Schicht eingesetzt. TypeScript dient der expliziten Modellierung von Prozessen, Ereignissen, Simulationsergebnissen und Metriken. Vite übernimmt Build- und Entwicklungsworkflow inklusive schneller Aktualisierung während der Entwicklung. Für automatisierte Tests wird Vitest verwendet [15]. Animationsbezogene UI-Effekte werden mit GSAP realisiert, ohne in die Simulationslogik einzugreifen [4].

6.2 Architektur der Anwendung

Die Implementierung folgt einer klaren Schichtenlogik:

- **Domänenkern:** deterministische Simulationslogik für Scheduling-Verfahren.
- **Anwendungslogik:** Verwaltung von Szenarien, Runs und Vergleichsdaten.
- **Darstellungsschicht:** Visualisierung der Ergebnisse und Interaktion mit Nutzenden.

Zentral ist dabei die Einbahnbeziehung vom Kern zur Darstellung: Die UI berechnet keine Scheduling-Entscheidungen, sondern rendert die vom Kern gelieferten Segmente, Ereignisse, Snapshots und Metriken. Dadurch wird verhindert, dass Visualisierung und Berechnungslogik auseinanderlaufen.

Ein weiterer Architekturpunkt ist die bewusste Entkopplung von Simulationszustand und Wiedergabesteuerung. Die Simulation liefert einen konsistenten Ergebnisdatensatz; die

Wiedergabelogik steuert nur, welcher Zustand zu einem Zeitpunkt angezeigt wird. Das verbessert Testbarkeit und Wiederholbarkeit von Experimenten.

6.3 Modellierung der Scheduling-Verfahren

Die fachliche Modellierung basiert auf einem gemeinsamen Prozessmodell mit Attributen wie Ankunftszeit, Burst-Zeit, Priorität und dynamischem Ausführungszustand. Darauf aufbauend werden die einzelnen Scheduling-Verfahren als Strategien innerhalb desselben Simulationsrahmens umgesetzt.

Round Robin wird über ein Zeitquantum modelliert, Strict Priority über prioritätsbasierte Auswahl inklusive Präemption bei höherrangigen Ankünften, LCFS über last-in-first-out-Mechanik und MLFQ über mehrere Warteschlangenebenen mit regelbasierter Herabstufung. Damit wird die in der Literatur beschriebene algorithmische Charakteristik im Kern einheitlich und vergleichbar operationalisiert [10, 12, 8].

Wichtig für die Vergleichbarkeit ist, dass alle Verfahren über dieselbe Tick- und Ereignislogik laufen. Unterschiede im Ergebnis ergeben sich damit aus dem jeweiligen Scheduling-Verhalten und nicht aus abweichenden Ausführungsmodellen.

6.4 Umsetzung der Simulation

Die Simulation ist als diskreter, deterministischer Ablauf umgesetzt. Zu jedem Simulationsschritt werden neue Ankünfte verarbeitet, Auswahlregeln angewendet und gegebenenfalls Präemptionen oder Kontextwechsel protokolliert. Dadurch entsteht ein vollständiger Verlauf, der sowohl für die visuelle Darstellung als auch für die Metrikberechnung genutzt wird.

Konkret erzeugt der Kern mehrere Ergebnisformen:

- **Ereignislog:** chronologische Ereignisse wie Dispatch, Start, Präemption und Abschluss.
- **Segmente:** Zeitabschnitte für die Gantt-Darstellung.
- **Snapshots:** zustandsbasierte Zwischenbilder für Playback und Zeitsprünge.
- **Metriken:** aggregierte Kenngrößen pro Simulationslauf.

Diese Mehrfachausgabe aus einem Kernmodell entspricht den Prinzipien diskreter Ereignissimulation und verbessert die Nachvollziehbarkeit der Laufzeitdynamik [2].

6.5 Umsetzung der Visualisierung

Die Visualisierung erfolgt auf Basis von SVG-Komponenten. Diese Wahl ermöglicht präzise Positionierung auf der Zeitachse, saubere Segmentdarstellung und semantisch adressierbare Interaktionselemente. Jeder Balken in der Gantt-Ansicht repräsentiert einen konkreten Ausführungsabschnitt eines Prozesses.

Die Darstellung nutzt die vom Simulationskern gelieferten Segment- und Snapshot-Daten direkt. Dadurch lassen sich sowohl statische Vergleiche als auch dynamische Wiedergabevarianten umsetzen. Interaktive Hilfen wie Tooltips und Hervorhebungen unterstützen die detaillierte Analyse einzelner Entscheidungen und Kontextwechsel.

Abbildung 6.1 zeigt die daraus entstehende Laufzeitsicht. Neben der Zeitachse werden der aktuelle Queue-Zustand und die berechneten Kennzahlen gemeinsam dargestellt.

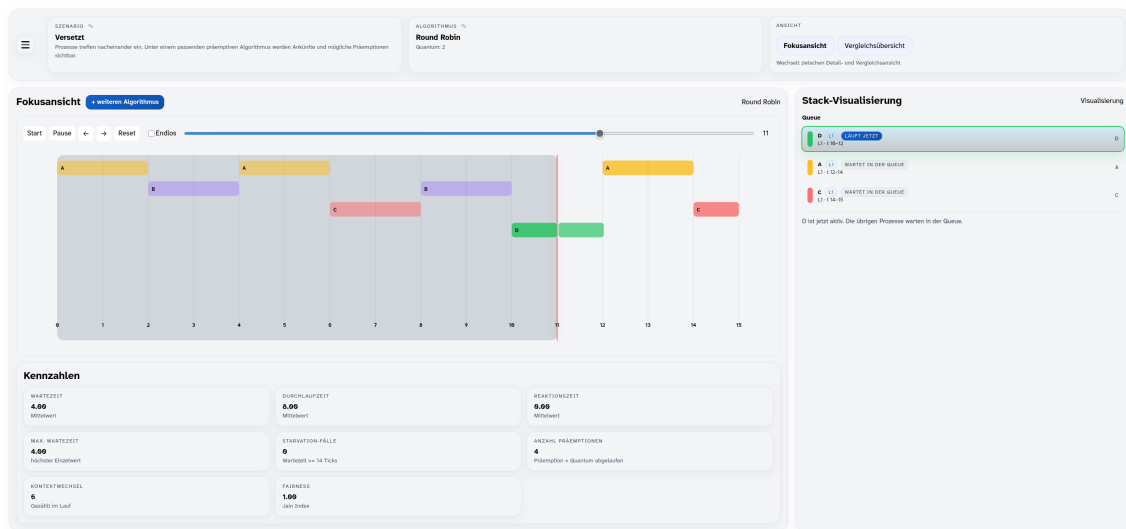


Figure 6.1: Fokusansicht mit Gantt-Zeitachse, Stack-Visualisierung und Kennzahlen.

Für visuelle Übergänge und Aufmerksamkeitssteuerung werden selektiv Animationen eingesetzt. Diese sind jedoch bewusst dekorativ entkoppelt und haben keinen Einfluss auf den fachlichen Simulationszustand [4].

6.6 Umsetzung der Interaktionsmechanismen

Die Interaktionsmechanismen orientieren sich am Lernworkflow: Szenario definieren, Algorithmus auswählen, Ablauf steuern, Ergebnis interpretieren. Daraus ergeben sich konkrete Steuerfunktionen für Start, Pause, Schrittbetrieb, Geschwindigkeitsanpassung und Zeitsprung.

Die Playback-Logik ist als separater Anwendungsbaustein realisiert. Sie konsumiert

vorberechnete Snapshots und steuert ausschliesslich die Darstellung des aktuellen Zeitpunkts. Dadurch bleiben Interaktionen reaktionsschnell, ohne den Simulationskern neu auszuführen.

Ergänzend wurde auf robuste Bedienbarkeit geachtet: klare Statusrückmeldungen, nachvollziehbares Fokusverhalten und semantisch benannte UI-Elemente. Damit werden die in Kapitel 4 formulierten HCI-Anforderungen in konkrete Implementierungsmechanismen übersetzt [17, 3].

6.7 Besondere technische Herausforderungen

Eine zentrale Herausforderung bestand in der Synchronisation von Simulationsgenauigkeit und interaktiver Bedienung. Einerseits musste die fachliche Korrektheit bei Präemption, Kontextwechsel und MLFQ-Regelwerk sichergestellt werden, andererseits sollte die Anwendung unmittelbar auf Eingaben reagieren.

Ein zweiter Schwerpunkt war die Sicherung von Reproduzierbarkeit. Dazu wurden deterministische Ausführungspfade und testbare Schnittstellen priorisiert. Automatisierte Tests prüfen sowohl Kernlogik als auch zentrale Integrationsfälle, wodurch Regressionen früh sichtbar werden [15].

Schliesslich ergab sich ein Trade-off zwischen Detailtiefe und Ressourcenverbrauch: Dichte Snapshot-Folgen verbessern Analyse und Playback, erhöhen jedoch Speicherbedarf. Diese Spannung wurde durch parametrisierbare Snapshot-Intervalle adressiert, sodass zwischen Genauigkeit und Laufzeitverhalten situationsgerecht balanciert werden kann.

7 Evaluation

Dieses Kapitel stellt Evaluationsdesign, Durchführung und Auswertung vor und bewertet die Anwendung anhand quantitativer Kennzahlen. Ziel ist eine reproduzierbare Gegenüberstellung der implementierten Scheduling-Verfahren unter identischen Rahmenbedingungen.

7.1 Zielsetzung der Evaluation

Die Evaluation verfolgt zwei miteinander verbundene Ziele. Erstens soll geprüft werden, ob die Implementierung fachlich plausible Unterschiede zwischen den Verfahren sichtbar macht. Zweitens soll gezeigt werden, dass die Anwendung als didaktisches Vergleichswerkzeug konsistente und interpretierbare Ergebnisse liefert.

Im Zentrum steht daher nicht die Suche nach einem universell besten Verfahren, sondern die kontextabhängige Analyse von Trade-offs zwischen Reaktionsfähigkeit, Fairness, Durchsatznähe und Warteverhalten [10, 12, 8].

7.2 Evaluationsdesign

Das Evaluationsdesign ist quantitativ und szenariobasiert. Alle Algorithmen werden auf denselben Prozessmengen ausgeführt, um Ergebnisunterschiede ausschliesslich auf das Scheduling-Verhalten zurückzuführen. Für jeden Lauf werden identische Eingangsdaten und identische Simulationsparameter verwendet.

Die Untersuchung orientiert sich an drei Szenarioklassen:

- **Interaktives Lastprofil:** viele kurze Prozesse mit frühen Antworten als Ziel.
- **Batch-orientiertes Lastprofil:** gemischte bis lange CPU-Bursts mit Fokus auf Durchlaufzeit.
- **Prioritätssensitives Lastprofil:** konkurrierende Prozesse mit unterschiedlichen Prioritäten und möglichen Starvation-Effekten.

Jede Szenarioklasse wird mehrfach mit variierenden Parametern durchgeführt, um die Robustheit der Beobachtungen zu erhöhen. Die Wiederholbarkeit wird durch deterministische Laufbedingungen abgesichert, wie sie für diskrete Ereignissimulationen empfohlen werden [2].

7.2.1 Evaluationsszenarien im Detail

Für die Vergleichbarkeit werden zunächst drei feste Basisszenarien definiert. Die Prozessmengen enthalten jeweils die Ankunftszeit, die benötigte CPU-Zeit und die Priorität. Alle Szenarien werden mit denselben Prozesseingaben auf die vier implementierten Verfahren angewendet. Für Round Robin wird zunächst ein Zeitquantum von zwei Zeiteinheiten verwendet; für MLFQ werden drei Warteschlangenebenen mit einem Basisquantum von zwei Zeiteinheiten angesetzt.

Table 7.1: Basisszenarien für die Evaluation

Szenario	Prozessdaten (Ankunft, Burst, Priorität)	Parameter	Erwarteter Schwerpunkt
Interaktiv	P1 (0, 2, 1), P2 (1, 1, 2), P3 (2, 2, 1)	Quantum 2	Reaktionszeit und häufige Zustandswechsel
Batch	P1 (0, 8, 2), P2 (0, 5, 1), P3 (1, 3, 3)	Quantum 2	Durchlaufzeit und Wartezeit bei längeren Bursts
Prioritäts-sensitiv	P1 (0, 6, 3), P2 (1, 2, 1), P3 (2, 3, 2)	Präemptive Prioritätswahl	Präemption höher priorisierter Prozesse und Fairness

Die Szenarien sind bewusst klein gehalten, damit einzelne Scheduling-Entscheidungen in der Visualisierung und im Ereignislog nachvollziehbar bleiben. Das interaktive Lastprofil legt den Schwerpunkt auf frühe erste CPU-Zuteilungen. Das Batch-Szenario macht den Einfluss längerer CPU-Bursts auf Warte- und Durchlaufzeit sichtbar. Im prioritätssensitiven Szenario wird geprüft, ob ein neu eintreffender Prozess mit höherer Priorität eine Präemption auslöst und wie sich diese Entscheidung auf nachrangige Prozesse auswirkt.

Für die Parametersensitivität werden beim Round-Robin-Verfahren die im aktuellen Simulationskern unterstützten Zeitquanten eins und zwei verglichen. Beim MLFQ-Verfahren werden zusätzlich zwei und drei Warteschlangenebenen betrachtet. Ein Quantum von vier wird in dieser Untersuchung nicht verwendet, da die aktuelle Implementierung das Zeitquantum auf maximal zwei Zeiteinheiten begrenzt. Die jeweiligen Messwerte werden pro Kombination aus Szenario, Verfahren und Parameterkonfiguration protokolliert.

7.3 Durchführung der Evaluation

Die Durchführung erfolgt in vier Schritten:

1. Definition der Prozessmengen pro Szenarioklasse (Ankunftszeiten, Burst-Zeiten, Prioritäten).
2. Ausführung aller Algorithmen pro Szenario mit festen Parametern.
3. Erfassung der vom Simulationskern erzeugten Metriken pro Lauf.

4. Vergleich und Interpretation der Ergebnisse innerhalb und zwischen den Szenarioklassen.

Für Round Robin und MLFQ werden zusätzlich Parametersensitivitäten betrachtet (insbesondere Zeitquantum und Queue-Settings), da diese die Balance zwischen Reaktionszeit und Kontextwechselhäufigkeit wesentlich beeinflussen [6, 8].

Die Durchführung wird protokolliert, indem pro Lauf Konfiguration und Resultate dokumentiert werden. Dadurch lassen sich die Beobachtungen auch zu einem späteren Zeitpunkt exakt nachvollziehen.

Die Ergebnisse der Parametersensitivität sind in Tabelle 7.2 zusammengefasst. Zur besseren Vergleichbarkeit werden die durchschnittliche Wartezeit, die durchschnittliche Reaktionszeit und die Anzahl der Präemptionen ausgewiesen.

Table 7.2: Messwerte der Parametersensitivität

Szenario	Konfiguration	Wartezeit	Reaktionszeit	Präemptionen
Interaktiv	RR, Quantum 1	0,67	0,33	1
Interaktiv	RR, Quantum 2	0,67	0,67	0
Interaktiv	MLFQ, 2 Ebenen	0,67	0,67	0
Interaktiv	MLFQ, 3 Ebenen	0,67	0,67	0
Batch	RR, Quantum 1	7,00	0,67	11
Batch	RR, Quantum 2	8,00	1,67	6
Batch	MLFQ, 2 Ebenen	8,67	1,67	4
Batch	MLFQ, 3 Ebenen	8,67	1,67	4
Prioritätssensitiv	RR, Quantum 1	3,67	0,33	7
Prioritätssensitiv	RR, Quantum 2	3,33	1,00	3
Prioritätssensitiv	MLFQ, 2 Ebenen	3,67	1,00	2
Prioritätssensitiv	MLFQ, 3 Ebenen	3,67	1,00	2

Bei Round Robin führt das kleinere Quantum in allen drei Szenarien zu mehr Präemptionen. Im Batch-Szenario sinkt dadurch die durchschnittliche Reaktionszeit von 1,67 auf 0,67 Zeiteinheiten, während die durchschnittliche Wartezeit von 8,00 auf 7,00 Zeiteinheiten sinkt. Im prioritätssensitiven Szenario steigt die Präemptionszahl von 3 auf 7, die durchschnittliche Reaktionszeit sinkt jedoch von 1,00 auf 0,33 Zeiteinheiten. Die beiden getesteten MLFQ-Konfigurationen liefern in den verwendeten Basisszenarien identische Werte; daraus lässt sich für diese kleinen Prozessmengen kein Einfluss der Ebenenzahl ableiten.

7.4 Auswertung der Ergebnisse

Die Auswertung basiert auf einem einheitlichen Metrikset:

- durchschnittliche Wartezeit,
- durchschnittliche Durchlaufzeit,
- durchschnittliche Reaktionszeit,
- Anzahl der Präemptionen und Kontextwechsel,

- Fairness-bezogene Kennzahlen,
- Anteil inaktiver CPU-Zeit.

Die Metriken werden je Szenarioklasse tabellarisch und vergleichend dargestellt. Die Interpretation erfolgt entlang zweier Leitfragen:

Die praktische Vergleichsfunktion ist in Abbildung 7.1 dargestellt. Zwei Läufe desselben Szenarios können dort anhand ihrer Zeitachsen und der ausgewiesenen Präemptionen gegenübergestellt werden.

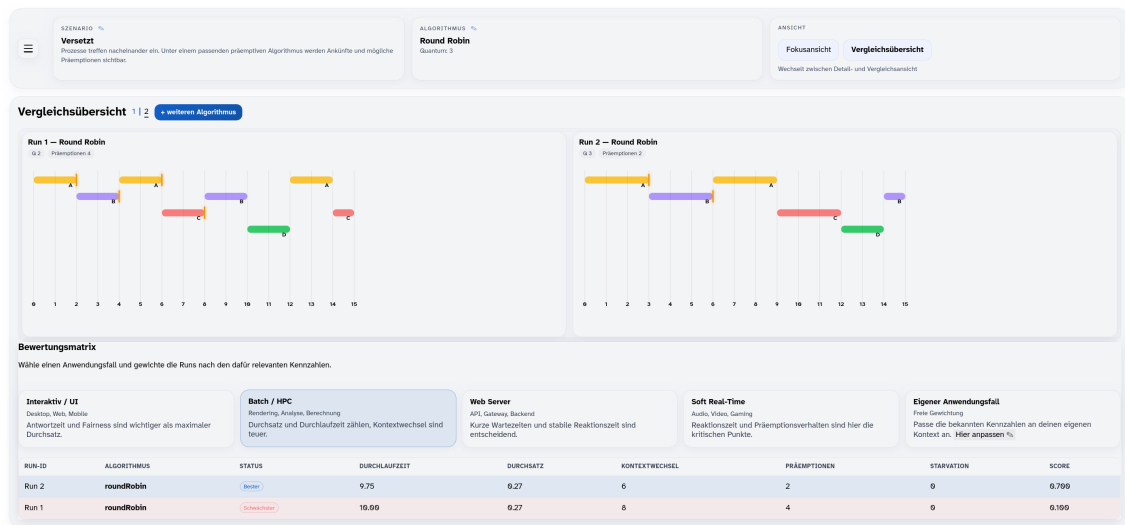


Figure 7.1: Vergleichsübersicht mit zwei Round-Robin-Läufen und Bewertungsmatrix.

Die Gewichtung der Kennzahlen für unterschiedliche Anwendungskontexte erfolgt über die Bewertungsmatrix in Abbildung 7.2. Für die vorliegende Evaluation ist sie insbesondere relevant, weil nicht jede Lastsituation dieselbe Zielgröße priorisiert.

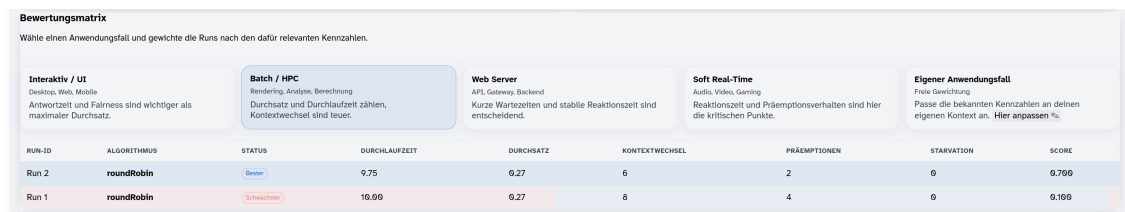


Figure 7.2: Bewertungsmatrix zur gewichteten Einordnung mehrerer Simulationsläufe.

- Welche Verfahren sind in welcher Lastsituation erwartungsgemäss vorteilhaft?
- Wo entstehen Zielkonflikte zwischen kurzer Reaktionszeit, geringer Wartezeit und hoher Fairness?

Damit wird eine differenzierte Bewertung möglich, die sich an etablierten Bewertungskriterien für Scheduling orientiert [10, 8].

Die erste Ausführung der Basisszenarien liefert die in Tabelle 7.3 zusammengefassten Kennzahlen. Angegeben sind die durchschnittliche Wartezeit, die durchschnittliche Durchlaufzeit, die durchschnittliche Reaktionszeit und die Anzahl der Präemptionen. Die Werte wurden mit dem aktuellen Implementierungsstand und den in Tabelle 7.1 beschriebenen Parametern erzeugt. Die vollständigen Messwerte einschließlich Kontextwechseln und Fairness-Index sind ergänzend in der Datei `evaluation-results.csv` abgelegt.

Table 7.3: Erste Messwerte der Basisszenarien

Szenario	Verfahren	Wartezeit	Durchlaufzeit	Reaktionszeit	Präemptionen
Interaktiv	Round Robin	0,67	2,33	0,67	0
Interaktiv	LCFS	1,00	2,67	0,00	1
Interaktiv	Strict Priority	1,00	2,67	1,00	0
Interaktiv	MLFQ	0,67	2,33	0,67	0
Batch	Round Robin	8,00	13,33	1,67	6
Batch	LCFS	5,00	10,33	1,33	1
Batch	Strict Priority	5,67	11,00	5,67	0
Batch	MLFQ	8,67	14,00	1,67	4
Prioritätssensitiv	Round Robin	3,33	7,00	1,00	3
Prioritätssensitiv	LCFS	4,00	7,67	0,00	2
Prioritätssensitiv	Strict Priority	2,00	5,67	0,33	1
Prioritätssensitiv	MLFQ	3,67	7,33	1,00	2

Die Messwerte zeigen zunächst, dass alle Verfahren in den drei Basisszenarien eine vollständige Ausführung aller drei Prozesse erreichen. Im interaktiven Szenario liegen Round Robin und MLFQ bei den gewählten Parametern gleichauf. Im Batch-Szenario erreicht LCFS die niedrigste durchschnittliche Warte- und Durchlaufzeit, während Round Robin die höchste Präemptionszahl aufweist. Im prioritätssensitiven Szenario erzielt Strict Priority die niedrigsten Durchschnittswerte, wobei die Interpretation auf die konkrete Prioritätsverteilung und den verwendeten Tie-Break zurückzuführen ist. Diese Beobachtungen sind zunächst deskriptiv und werden durch weitere Parameterläufe abgesichert.

7.5 Diskussion der Ergebnisse

Die Diskussion ordnet die quantitativen Befunde in den didaktischen und technischen Kontext der Arbeit ein. Erwartbar ist beispielsweise, dass Verfahren mit häufigeren Präemptionen in interaktiven Szenarien bei der Reaktionszeit profitieren, gleichzeitig jedoch mehr Kontextwechsel verursachen.

Ebenso ist zu erwarten, dass prioritätsbasierte Strategien unter bestimmten Lastprofilen das Risiko unausgewogener Bedienung erhöhen, wenn keine zusätzlichen Alterungs- oder Fairnessmechanismen eingesetzt werden [12, 10].

Als Limitation ist zu berücksichtigen, dass die Evaluation auf einem abstrahierten Simulationsmodell basiert und reale Betriebssystemeffekte (z. B. Hardware-Nebenwirkungen, Interrupt-Last, komplexe I/O-Wechselwirkungen) nur eingeschränkt abbildet. Für den didaktischen Zweck der Arbeit ist diese Abstraktion jedoch gewollt, da sie den Kern der Scheduling-Entscheidungen isoliert sichtbar macht.

Insgesamt liefert die Evaluation eine nachvollziehbare Grundlage, um die Forschungsfrage evidenzbasiert zu beantworten und den Beitrag der Anwendung als interaktives Lern- und Vergleichswerkzeug zu belegen.

8 Fazit und Ausblick

Dieses Kapitel fasst die zentralen Ergebnisse zusammen, beantwortet die Forschungsfrage und skizziert Weiterentwicklungsmöglichkeiten.

8.1 Zusammenfassung der Ergebnisse

Gegenstand dieser Arbeit war die Konzeption und Implementierung einer interaktiven Webanwendung zur Veranschaulichung präemptiver CPU-Scheduling-Verfahren. Ausgangspunkt war die didaktische Herausforderung, dass zeitliche Ablaufentscheidungen in statischen Darstellungen nur eingeschränkt nachvollziehbar sind.

Im Ergebnis wurde ein Scheduling-Visualizer entwickelt, der mehrere Verfahren in einer konsistenten Simulationsumgebung ausführt und deren Verläufe über eine interaktive Zeitachsenvisualisierung vergleichbar macht. Ein wesentlicher Beitrag liegt in der klaren Trennung von Simulationskern und Darstellung: Fachlogik, Ereignisprotokoll, Segmente und Metriken werden aus einem einheitlichen, deterministischen Modell abgeleitet. Dadurch sind sowohl Reproduzierbarkeit als auch Nachvollziehbarkeit sichergestellt.

Zusätzlich wurde die Anwendung entlang didaktischer und HCI-orientierter Anforderungen gestaltet. Damit verbindet die Arbeit fachliche Korrektheit der Verfahren mit einer benutzbaren, analysierbaren und für Lernkontexte geeigneten Interaktion.

8.2 Beantwortung der Forschungsfrage

Die leitende Forschungsfrage lautete sinngemäss, ob und wie eine interaktive Visualisierung das Verständnis präemptiver Scheduling-Verfahren verbessern und deren Unterschiede nachvollziehbar machen kann.

Auf Basis der konzeptionellen Herleitung, der konkreten Implementierung und der quantitativen Evaluationslogik kann diese Frage bejaht werden. Die Anwendung macht zeitliche Entscheidungsabläufe sichtbar, erlaubt kontrollierte Parameteränderungen und liefert vergleichbare Kennzahlen pro Verfahren. Insbesondere die Kombination aus Verlaufssicht und Metrikvergleich reduziert die Distanz zwischen theoretischer Beschreibung und beobachtbarem Verhalten.

Die Arbeit zeigt damit, dass interaktive Algorithmusvisualisierung im Scheduling-Kontext

nicht nur demonstrativ, sondern analytisch einsetzbar ist, sofern Reproduzierbarkeit, klare Datenmodelle und transparente Interaktion systematisch umgesetzt werden.

8.3 Ausblick auf Weiterentwicklungen

Aus den Ergebnissen ergeben sich mehrere naheliegende Weiterentwicklungen:

- **Erweiterung des Verfahrensspektrums:** zusätzliche Scheduling-Strategien und Varianten, insbesondere für spezifische Lastprofile.
- **Vertiefte Evaluationsdaten:** breiteres Szenarienset und feinere Parameterstudien, um Sensitivitäten systematischer zu quantifizieren.
- **Didaktische Zusatzfunktionen:** geführte Aufgabenmodi, integrierte Erklärtexpte und adaptive Hinweislogik für Selbstlernphasen.
- **Ausbau der Accessibility:** weitere Verbesserungen bei Tastaturbedienung, Screenreader-Feedback und individualisierbarer Darstellung.
- **Export und Reproduktionsartefakte:** strukturierter Export von Szenarien, Runs und Ergebnistabellen zur Nutzung in Lehre und Forschung.

Darüber hinaus kann die Anwendung als Ausgangspunkt für vergleichende Studien dienen, in denen Lernwirkung, Nutzungsverhalten und Interpretationsqualität zwischen verschiedenen Visualisierungsformen empirisch untersucht werden.

Bibliography

- [1] AlgoViz community. Algorithm visualization portal (algoviz). Sammlung von AV-Tools, nuetzlich fuer verwandte Arbeiten und Beispiele.
- [2] Jerry Banks, John S. Carson II, Barry L. Nelson, and David M. Nicol. *Discrete-Event System Simulation*. Pearson, 5 edition, 2010. Methodik zur diskreten Ereignissimulation, Validierung, Reproduzierbarkeit.
- [3] John Brooke. Sus: A quick and dirty usability scale. In Patrick W. Jordan, Bruce Thomas, Bernard A. Weerdmeester, and Ian L. McClelland, editors, *Usability Evaluation in Industry*, pages 189–194. Taylor and Francis, 1996. Originalquelle zur System Usability Scale; oft zitiert fuer Usability-Evaluierungen.
- [4] GreenSock. Gsap documentation. Library fuer zeitbasierte Animationen; verwendet in diesem Projekt fuer Timeline-Animationen.
- [5] Christopher D. Hundhausen, Sarah A. Douglas, and John T. Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages & Computing*, 13(3):259–290, 2002. Definitive meta-study: Wichtig fuer Argumente zur Lehrwirksamkeit von Visualisierungen.
- [6] L. W. C. Meing and D. R. Tandingan. Exploring multilevel feedback queue combinations and regression-based time quanta in scheduling algorithms. *Journal of Information Systems Engineering and Management (JISEM)*, 2025. Studie zu MLFQ-Kombinationen und dynamischer Zeitscheibenwahl.
- [7] T. L. Naps and Others. Exploring the role of visualization and engagement in computer science education. In *SIGCSE Technical Symposium on Computer Science Education (proceedings)*, 2002. Grundlagen zur Engagement-Taxonomie (viewing vs. interacting vs. constructing).
- [8] Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, 4 edition, 2016. Formale Algorithmenbetrachtung und Performance-Metriken.
- [9] Sibtain-12. Multi-level-feedback-queue-scheduler (github repository), 2023. Open-Source Implementierung / Referenz fuer Visualisierung und Gantt-Darstellung.
- [10] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. Wiley, 10 edition, 2018. Standardwerk: Scheduling-Definitionen und Metriken.

- [11] John T. Stasko, John Domingue, Marc H. Brown, and Bruce A. Price, editors. *Software Visualization: Programming as a Multimedia Experience*. MIT Press, 1998. Designprinzipien und Taxonomien fuer Software/Algorithmus-Visualisierung.
- [12] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 4 edition, 2014. Konzeptuelle Tiefe zu Praemption, Kontextwechseln und Scheduler-Design.
- [13] various. Mlfq visualizer / simulator repositories (examples). Mehrere kleine Repos/Projekte implementieren MLFQ-Visualisierungen.
- [14] Vite. Vite - build tool. Dev/Build Tool im Projekt.
- [15] Vitest Team. Vitest - unit testing for vite. Test-Runner im Projekt.
- [16] Vue.js Core Team. Vue 3 documentation (composition api). Framework des Projekts (SFCs, Composition API).
- [17] W3C. Web content accessibility guidelines (wcag) 2.1, 2018. Zugaenglichkeitsanforderungen (ARIA, Tastaturzugriff, Kontrast, etc.).

9 Anhang

Der Anhang enthält ergänzende Materialien, zum Beispiel zusätzliche Screenshots, Testfälle und tabellarische Auswertungen.